



The impact of time: lifetime as governing reward in Reinforcement Learning

Candidate number: BRZJ7 ¹

MSc Data Science and Machine Learning

Supervised by:
Dr Laura Toni
Eduardo Pignatelli

September 2023

¹**Disclaimer:** This report is submitted as part requirement for the MSc Data Science and Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged

Abstract

Reward shaping is a technique in reinforcement learning to solve hard credit assignment problems. Yet, its reliance on specific domain knowledge makes it complex and time-consuming to do it for various problems. In this study, we investigate the idea of a universal reward shaper: time. We divide the research into two different methodologies: firstly, we examine how time could be used as an intrinsic reward, with results suggesting a bigger potential for using time as the sole reward. In our second approach, we train agents in the absence of standard extrinsic rewards, under the assumption that termination probabilities inherently encode the goals of the problem, an idea observed in many real-world settings. Results show that time-based agents can come close to or even surpass agents trained in the traditional setting both in end performance and learning speed. Our findings highlight the significance and potential of an agent’s lifetime in reward design but with significant challenges that need to be overcome. Further research is needed to validate our methodologies in diverse and more complex environments and to explore ways of generalising this idea to a broader spectrum of reinforcement learning domains.

Acknowledgements

I would like to thank my thesis supervisors Laura and Edu for the project proposition and their guidance, advice and valuable inputs throughout the project. Every discussion was helpful and ensured I remained on the right path. I also want to acknowledge the science library where this research - among many others - took shape after long and long hours. And lastly, special thanks to my family and friends for being always there for me.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	4
1.3	Proposed approach and contributions	5
1.4	Thesis structure	5
2	Background	7
2.1	Reinforcement Learning overview	7
2.2	Deep reinforcement learning	12
2.3	Policy gradient methods	13
2.3.1	Policy Gradient Methods	13
2.3.2	Actor-Critic methods	15
2.3.3	Proximal Policy Optimization	15
2.4	The role of rewards	17
2.4.1	Intrinsic motivation	18
2.4.2	Reward shaping	18
3	Related work	20
4	Soft approach: Reward Shaping with Time	23
4.1	Methodology	23
4.2	Experiment I	25
4.2.1	Experimental setup	25
4.2.2	Experiment design	26
4.3	Results and discussion	28
5	Hard Approach: Lifetime as Reward	33
5.1	Methodology	33
5.2	Experiment II	35
5.2.1	Experimental setup	35

5.2.2 Experiment design	35
5.3 Results and discussion	38
6 Conclusions and Future Work	42
List of Figures	44
Bibliography	45

Chapter 1

Introduction

1.1 Motivation

A reinforcement learning agent aims to learn how to act in certain situations through interaction with an environment, which gives a reward as a response to each action. By acting differently in various situations the agent learns to choose those actions which provide the highest reward in the long run. The concept of *learning through interaction* is the distinctive feature that distinguishes reinforcement learning from the other machine learning paradigms.

This approach to learning has led to groundbreaking results, making it the state-of-the-art technique for many machine learning tasks. From well-known historic successes like AlphaGo to daily use cases such as content recommendations on social media or the development of large language models, namely, ChatGPT, reinforcement learning is driving the progress of artificial intelligence at an unprecedented pace. There are constant additions to literature which not only improve performance and increase robustness but also broaden its applicability across numerous fields.

The design of reinforcement learning tasks, however, poses significant challenges that may hinder its potential. A key issue, and the one we will address in this study, is the necessity of task-specific knowledge in reward design. Reward entirely determines the behaviour of an agent and, as the driver of the learning process, designing a good reward function is crucial. In some cases, a straightforward reward function that intuitively describes what we want the agent to do might not be enough to guide the agent towards the desired behaviour. For example, consider a hypothetical environment where the agent needs to reach a goal, navigating through doors, enemies and other hazards along the path. Although giving a positive reward only when the agent reaches the goal describes exactly the task we want the agent to solve, some of the current RL methods might struggle to leverage it effectively. In those scenarios, without any specific exploration strategy, re-

warding sub-goals that guide the agent towards the main goal can significantly improve performance.

The technique proposed for the previous example, known as *reward shaping*, is a common practice in reinforcement learning. This technique will be formalised in later chapters, but in a few words, it involves modifying the original reward function in a way that directs the agent toward certain desirable behaviours without altering the underlying task objectives. Despite its effectiveness, this requires having a deep understanding of the dynamics of the environment, resulting in specific solutions that may not generalise well to different tasks. Moreover, as we look to apply reinforcement learning in increasingly complex, real-world tasks, the difficulty of applying reward shaping increases even more, due to its lack of scalability: it needs to be done for every task we want the agent to solve. All of these make reward shaping a challenging technique to apply in some cases.

We might then wonder whether we can set up the reinforcement learning problem such that the reward function does not depend on the environment. If the answer to this question is affirmative, we could exploit this framework to train reinforcement learning agents without the need to specify the rewards. This would eliminate the need for task-specific knowledge, helping to make reinforcement learning more universally applicable across various tasks and environments.

In this study, we investigate the idea of using time as reward in response to that question, offering a potential approach to task-dependent reward design challenges. The survival time of the agent is a general and task-agnostic value that, in many cases, can somewhat describe the performance of the agent. Encouraging longer or shorter lifetimes can motivate the agent to learn certain desired behaviours, as survival dynamics often implicitly encode goals in the environment. For that reason, time can be a good reward shaper that does not involve the common issues when designing that technique. More ambitiously, in some environments, time may encode enough information to potentially serve as the only reward, removing any traditional reward, whose design, again, comes with the aforementioned challenges.

1.2 Objectives

The main objective of this study is to explore whether lifetime can serve as a reward (or part of it), offering a viable approach to design reinforcement learning problems that do not make use of task-specific knowledge. This would reduce the need for redesigning or fine-tuning reward systems for every task. Compared to a traditional shaper, this would provide a more adaptable and generalizable framework which could be used in a variety of tasks, allowing for greater scalability of reinforcement learning to real-world tasks. To this end, we investigate the following research questions:

- **RQ1:** Can the concept of *lifetime as reward* be used to solve reinforcement learning problems?
- **RQ2:** What is the effect of *lifetime as reward* on the performance and behaviour of the agent?
- **RQ3:** Can the lifetime-as-reward framework be broadly applied to reinforcement learning problems? If so, in which type of settings may it be useful?

1.3 Proposed approach and contributions

To address these questions, we propose a study that comprises two different methodologies, which we define as the *soft* and *hard* approaches.

The first approach explores whether the incorporation of time can enhance the learning process by using it as a reward shaper. We call it “soft” because it does not remove any information from the original MDP, it just adds the time information on top of the original reward function. Our experiments showed that time can indeed be employed in reward shaping and the effect on the learning process is noticeable. The main contribution of this approach is to present a way to use time for reward shaping without modifying the classical MDP setting. Supported by the positive results obtained in the soft approach, we moved to our second method. Here, time serves as the only reward, with the assumption that the task is naturally encoded in the survival dynamics, which are represented by the probability of a state being absorbing (i.e., death), effectively guiding the agent in the absence of other traditional rewards. This is a more radical view where time acts as the sole reward and we remove the task information from the original reward function. For that reason, we call this the “hard” approach. Our findings here show that agents have the potential to learn without the need for explicit task-oriented rewards in environments.

These methods represent two different perspectives on integrating the agent’s lifetime into the learning process, and due to their different natures and scopes, they could be treated as two separate studies. However, both the soft and hard approaches contribute significantly to the understanding of how an agent’s lifetime impacts the learning process. Therefore, the incorporation of both approaches allows us to gain a deeper comprehension of the effects of time on the learning process of reinforcement learning agents.

1.4 Thesis structure

This thesis is structured in the following chapters.

- **Chapter 2:** Provides the essential background relevant to the project. It offers an overview of reinforcement learning with a special focus on the role of rewards and

Proximal Policy Optimisation, the chosen learning algorithm.

- **Chapter 3:** Reviews previous studies on reward design and shaping, intrinsic rewards, and policy gradient methods.
- **Chapter 4:** Covers the soft approach end-to-end. This chapter outlines the proposed methodology, describes the designed experiment for this approach, and presents the results and findings.
- **Chapter 5:** Addresses the hard approach following the same structure as Chapter 4.
- **Chapter 6:** Outlines the conclusions drawn from the project, the limitations of the study, and possibilities for future work.

The code for this project can be found in <https://github.com/BRZJ7/msc-project>

Chapter 2

Background

This chapter aims to provide an overview of the key concepts and definitions of Reinforcement Learning and Markov Decision Processes (MDP), establishing the foundation and notation for the research project. The definitions and notations chosen mostly follow the ones proposed by Sutton and Barto (2018). Then we emphasize the essential role that rewards play in controlling an agent’s learning process including the distinction and interaction between intrinsic and extrinsic rewards and their significance in RL tasks. Finally, we review policy optimization and policy gradient methods, with a particular focus on Proximal Policy Optimization (PPO) algorithm, used in the conducted experiments.

2.1 Reinforcement Learning overview

The reinforcement learning paradigm aims to solve learning problems by interacting with an environment. In this setting, the model is instead an *agent*, which interacts with an *environment* by making decisions (*actions*). The agent receives a representation of the state of the environment (*state*) before taking each action. Upon an action is taken, the environment is affected, resulting in a transition to a new state and returning a feedback (*reward*) to the agent. This feedback is what guides its learning process, as the final goal of the agent is then to learn a behaviour (*policy*) which maximises the cumulative sum of rewards over time (*return*).

Markov Decision Processes (MDPs) A classical formalisation of this problem is through MDPs, which mathematically model the sequential decision-making problem and allow us to solve reinforcement learning problems using an idealised framework. Formally, an MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, \gamma)$ where

- $\mathcal{S}$ is a finite set of all possible states
- $\mathcal{A}$ is a finite set of all possible actions

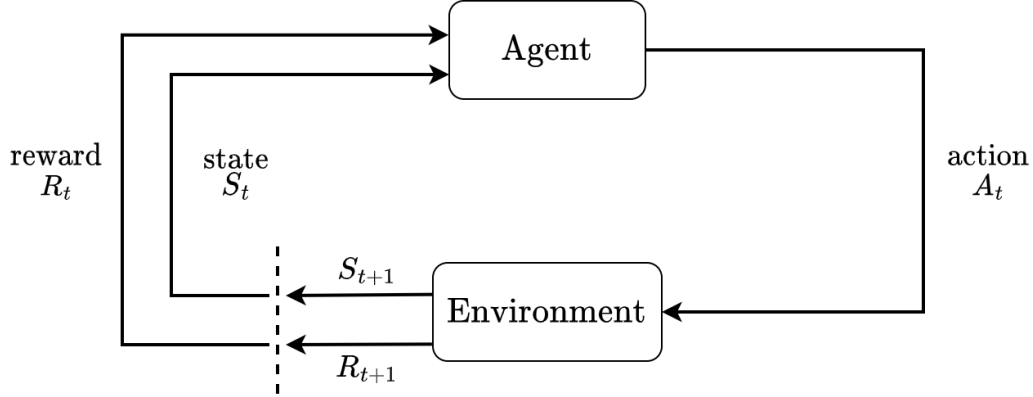


Figure 2.1: The agent-environment interaction in reinforcement learning. Image source: (Sutton and Barto (2018))

- $p(r, s' | s, a)$ is a transition probability operator which models the probability of obtaining a scalar reward $r \in \mathcal{R} \subset \mathbb{R}$ and transitioning to state s' , given the state-action pair (s, a)
- $\gamma \in [0, 1]$ is the discount factor, a scalar parameter that controls the trade-off between immediate and later rewards

The function p governs the dynamics of the problem and defines a probability distribution:

$$\sum_{r \in \mathcal{R}} \sum_{s' \in \mathcal{S}} p(r, s' | s, a) = 1 \quad \text{for all } s \in \mathcal{S}, a \in \mathcal{A} \quad (2.1)$$

The interaction of an agent with the environment then produces a sequence of states, actions and rewards called *trajectory* or *history* indexed by timesteps:

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots \quad (2.2)$$

At a given timestep t , the probability of the next state S_{t+1} being s' is defined by:

$$Pr\{S_{t+1} = s' | S_t = s, A_t = a\} = p(s' | s, a) = \sum_r p(r, s' | s, a) \quad (2.3)$$

Similarly, given a state $S_t = s \in \mathcal{S}$ and an action $A_t = a \in \mathcal{A}$, the expectation of the immediate reward R_{t+1} is given by

$$\mathbb{E}[R | s, a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(r, s' | s, a) \quad (2.4)$$

In this study, we will focus on deterministic reward functions, and thus we can write the

immediate reward R_{t+1} as a deterministic function of the current state $S_t = s$ and chosen action $A_t = a$:

$$R_{t+1} = r(s, a) \quad (2.5)$$

Equations 2.4, 2.3 then define the dynamics of the Markov Decision Process.

It is sometimes useful to define MDPs by decoupling the reward from the transition probability. In this representation, the MDP is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$ where all the components remain the same but $p(s'|s, a)$ models the transition only to state s' given the state-action pair (s, a) and $r(s, a)$ is a reward function that gives the expected immediate reward for taking action a in state s . We will use this representation when focusing on MDPs with modified reward functions.

A key assumption of MDPs is that the current state completely characterises the process. Formally, we say a state s has the *Markov property* when for states $\forall s' \in \mathcal{S}$

$$p(S_{t+1} = s' | S_t = s) = p(S_{t+1} = s' | h_{t-1}, S_t = s) \quad (2.6)$$

for all possible previous trajectories $h_{t-1} = \{S_1, \dots, S_{t-1}, A_1, \dots, A_{t-1}, R_2, \dots, R_t\}$. In an MDP, all states are assumed to satisfy the Markov property. This means that the current state captures all the relevant information and thus future states depend only on the current state and action, not on the sequence of actions and states that preceded it. This property is also known as *fully observability* and MDPs sometimes called fully-observable MDPs.

Agent learning process The agent interacts with this framework by choosing actions, which are selected according to a *policy*, denoted as π . This is a mapping $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ which for a given state s , assigns for each action $a \in \mathcal{A}$ the probability of taking that action, denoted by $\pi(a|s)$. The policy is subject to change over time as the agent refines its behaviour in response to interaction with the environment. This agent's adaptation is considered learning in the reinforcement learning paradigm.

In general, we seek to maximise the *expected return*. The (*discounted*) *return* at a time step t , denoted as G_t , is the sum of discounted future rewards, defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T \quad (2.7)$$

where T is a final timestep and γ is the discount factor. The use of the discount factor γ places more importance on immediate rewards and less on distant future rewards, and this effect increases as γ approaches 0.

This setting fits where the agent-environment interaction happens in consecutive finite sequences of time, called *episodes*, each with its own final timestep T . However, there are also situations where this interaction does not break naturally into identifiable episodes but

goes on continually without limit. This is referred to as the *continuing task* setting, where T is effectively infinity ($T = \infty$). For these situations, the discounted return is defined as an infinite sum:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.8)$$

Despite the episodic and continuing tasks being conceptually different, they can be treated within the same expected return maximisation framework. This is possible due to the use of discounting with $\gamma \in [0, 1)$, which makes sure that even in the infinite horizon case the return does not become infinite.

In either setting, the main goal of the agent is to maximise the average discounted return from each state, and this is achieved by finding a policy that leads to actions yielding the highest possible return. Formally, the maximisation objective is to find the policy that achieves the highest expected return from every state, denoted *optimal policy* π^* :

$$\pi^* = \max_{\pi} \mathbb{E}_{\pi}[G_t | S_t = s] \quad (2.9)$$

For MDPs, the existence of at least an optimal policy is guaranteed, but it might not be unique. Finding an optimal policy or the best possible approximation to one of those is often the main goal in reinforcement learning.

Value functions An important concept involved in this project and most of RL learning algorithms is *value functions*. Value functions estimate the expected cumulative reward an agent can achieve given a state and following a particular policy. They are essential components in the structure of many learning algorithms like, including Proximal Policy Optimization (PPO), which we will discuss later in this chapter. These functions provide a method for the agent to evaluate its current performance and guide its learning process towards improved policies.

The *value function* of a state s under a policy π , denoted as $v_{\pi}(s)$, is the expected return from state s when following policy π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] \quad (2.10)$$

Similarly, the *action-value function* of a state-action pair (s, a) , denoted as $q_{\pi}(s, a)$, is the expected return after taking action a in state s and thereafter following policy π :

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] \quad (2.11)$$

While both equations share the same conceptual underpinning, the first one relies on a

state and policy π thereafter, while the latter uses a given action to take the first step before adhering to π . Both formulations are commonly used, and the choice between them may be determined by the specifics of the situation at hand. In this report we might use the term value function to refer to either. The optimal state-value and action-value functions $v^*(s), q^*(s, a)$, respectively, are the maximum value functions over all possible policies. They are defined as:

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

Solving an MDP is equivalent to finding the optimal value function v^* (or q^*), because once it is known, the optimal policy π^* can be easily determined by choosing the action that yields the highest value at each state.

Partially-observable MDPs (POMDPs) The full-observability assumption introduced for MDPs might not always be true. In some situations, the agent may not have complete or perfect information about the current state of the environment, leading to situations where the agent must make decisions based on incomplete or uncertain information. POMDPs are MDPs in which the agent does not have access to the true state directly, but an *observation* instead, which contains some information about the underlying state, potentially incomplete or corrupted.

Formally, a POMDP is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma, \Omega, O)$ where

- $\mathcal{S}, \mathcal{A}, p, r$, and γ are the same as in MDPs
- Ω is a finite set of all possible observations
- O is the observation probability function, defined as $O(o|s')$, which models the probability of obtaining an observation $o \in \Omega$, given the true next state s'

The dynamics of POMDPs can be defined similarly to MDPs: at each timestep t , when an agent takes an action A_t in state S_t , the underlying state will transition to a new state S_{t+1} according to the transition probability function $p(s'|s, a)$. Simultaneously, the agent obtains an observation $O_t \in \Omega$, which is drawn from the distribution $O(o|s')$, defined by the observation function O . This means the probability of observing $O_t = o$ given the transition to the state $S_{t+1} = s'$ and action $A_t = a$ is $O(o|s')$.

The agent does not have direct access to the true state S_{t+1} ; it only has access to the (partial) observation O_t to decide on the next action A_{t+1} . Moreover, POMDPs do not necessarily satisfy the Markov property for which the state S_t contains all the information from history. The partial observability of states and lack of complete history knowledge brings

additional uncertainty and complexity to the decision-making process, making POMDPs significantly more difficult to model and solve.

2.2 Deep reinforcement learning

The foundations of reinforcement learning were established for tabular methods, where the state-action space is discrete and of limited size. In these scenarios, we can just learn an estimate for each state-action pair so that the agent chooses the action with the highest estimated value. Algorithms like Q-Learning (Watkins and Dayan (1992)) or SARSA (Rummery and Niranjan (1994)) are based on this principle, updating the estimate in various manners. However, in most real-world scenarios, where the action space is continuous and thus discretised to a large number of states, using tabular methods is infeasible. This is where neural networks, as the state-of-the-art function approximators of today, come into play in modern RL. Deep neural networks can be used to learn the value functions or policies for continuous and high-dimensional state spaces efficiently, what is known as *deep reinforcement learning*.

Deep reinforcement learning leverages the generalisation power of neural networks to learn complex representations of states and serve as function approximations. We can categorise deep RL into two groups based on what the network predicts. The first group is *value-based methods*, which use a network to predict the value of a given state-action pair. Based on this estimation, the agent will subsequently choose the action associated with the highest value. The second category is *policy-based methods*. In this approach, the neural network directly maps states to action distributions, determining the optimal action based on the learned policy rather than a value estimation. Both methods have advantages and disadvantages making them suitable for different settings. Figure 2.2 shows a depiction of these two methodologies. As we will see later in this chapter, some algorithms even combine both methods to leverage the strengths of each methodology in one single approach.

One of the fundamental contributions to value-based methods was Deep Q-Network (DQN), introduced by Mnih et al. (2013), which proposed to approximate the Q-learning function with a deep neural network. This outperformed human performance and state-of-the-art methods on Atari game environments and paved the way for using networks for RL problems. Several extensions of DQN followed, including Double DQN (van Hasselt et al. (2015)), which uses two value estimations to reduce overestimation bias; Dueling DQN (Wang et al. (2016)), that splits the representation of state values and action advantages for increased learning stability; and Rainbow DQN (Hessel et al. (2017)), which combined several of these enhancements into one method. Additional techniques that further improved the performance of these algorithms were introduced, such as experience replay (Wang et al. (2017)), to prioritise the most important past trajectories for learning and

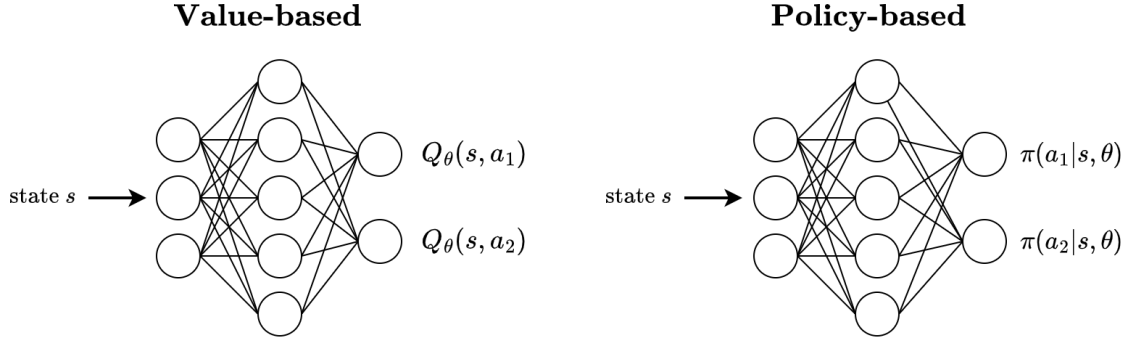


Figure 2.2: Comparison of value-based and policy-based deep reinforcement learning. For value-based methods, given a state s , the network predicts a value estimate $Q(s, a)$ for each of the actions a_1, a_2 . In the case of policy-based, the network directly outputs the policy in the form of a probability distribution over the actions, given the state s .

thus sample more efficiently. However, these methods, which learn an approximation of the value function and act greedily based on that, often result in discontinuous policies and struggle in scenarios where the optimal policy is stochastic.

Policy-based methods alleviate some of these issues: they can learn stochastic policies, which may be the only optimal policy in some settings and can learn smooth policy distributions, which usually leads to more robust learning. Given these advantages and the fact that the learning algorithm for our experiments falls into this category, we will now focus on policy optimization methods (often used interchangeably with policy-based methods).

2.3 Policy gradient methods

Policy gradient methods, within the family of policy-based methods, is a technique where the agent’s policy is directly optimized to maximize performance. This family of approaches includes Proximal Policy Optimization (PPO) (Schulman et al. (2017b)). As the chosen learning algorithm for the experiments conducted in this project, we will provide an overview of policy gradient methods and then delve deeper into the specific enhancements brought by PPO in the following subsections. From now on we consider *parameterised policies*, which are policies that are defined by a set of parameters θ . The policy with parameters θ is denoted π_{θ} .

2.3.1 Policy Gradient Methods

Policy gradient methods compute an estimator of the gradient of some performance measure (typically expected return) with respect to the policy parameters. Then a stochastic gradient ascent method is used to update those parameters and thus maximise the performance

of the parameterised policy.

Mathematically, this can be formulated as follows. Let θ be the parameters of policy π_θ and $G_t|_{\pi_\theta}$ the expected return when following policy π_θ . The objective of policy gradient methods is:

$$\theta^* = \arg \max_{\theta} G_t|_{\pi_\theta} \quad (2.12)$$

The main idea is that policy parameters are adjusted in the direction that maximizes expected return under the current parameterised policy, following gradient ascent principles. The parameters can be updated iteratively using the following equation:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} G_t|_{\pi_\theta} \quad (2.13)$$

where θ_{t+1}, θ_t are the value of parameters θ at timesteps $t + 1$ and t , respectively, $\hat{G}_t$ is an estimator of the expected return G_t and α is the learning rate. Doing this, the action probabilities are moved proportionally to the gradient.

A common parameterisation for policies in discrete action spaces is *action preferences*, in which each action-state (s, a) is parameterised by a scalar real value $h(s, a, \theta)$. Action preferences might be the parameters themselves or might be computed by a deep ANN where θ is the set of weights of the network. These action preferences then relate to the policy using a softmax distribution

$$\pi(a|s, \theta) = \frac{e^{h(s, a, \theta)}}{\sum_{a' \in \mathcal{A}} e^{h(s, a', \theta)}} \quad (2.14)$$

where e is the base of the natural logarithm, and the denominator ensures that the action probabilities sum to one in each state s .

To update the policy parameters efficiently, the *policy gradient theorem*, first introduced by (Sutton et al. (2000)), states that policy gradients do not need the MDP dynamics to compute the gradients and thus to update the policy, but only the return is needed. This theorem is essential to policy gradient methods as it offers a way to estimate the policy gradient without the need for a model of the environment. Therefore, we can estimate the gradient just by generating trajectories with the current policy

THEOREM 1 *For any differentiable policy $\pi_\theta(s, a)$, let d_0 be the starting distribution over states in which we begin an episode. Then, the policy gradient of $J(\theta) = \mathbb{E}[G_0 \mid S_0 \sim d_0]$ is*

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t q_{\pi_\theta}(S_t, A_t) \nabla_{\theta} \log \pi_\theta(A_t|S_t) \mid S_0 \sim d_0 \right] \quad (2.15)$$

where $q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$ is the value of the action-state (s, a) .

2.3.2 Actor-Critic methods

Actor-Critic methods split the roles of policy and value function into two components: the *actor*, which is responsible for choosing actions and the *critic*, which estimates the value of those actions.

The key idea is to maintain two separate parameterised functions: the policy $\pi_\theta(a|s)$ (actor) and the value function $v_\phi(s)$ (critic). The actor decides which action to take given the current state of the environment, while the critic evaluates the expected return from a given state according to the current policy. The associated parameters θ and ϕ are learnt simultaneously but separately. Typically, the learning process alternates between stages of policy evaluation, where the critic estimates the value function according to the current policy, and policy improvement, where the actor parameters are updated in the direction that the value function of the critic indicates.

More formally, the objective functions are defined as follows. For the critic:

$$L^{VF}(\phi) = \mathbb{E}_t [(v_\phi(s_t) - G_t)^2] \quad (2.16)$$

where G_t is the observed return from state s_t . The critic aims to minimize the discrepancy between the estimated value function and the observed returns. L^{VF} refers to value-function loss.

For the actor:

$$L^{actor}(\theta) = \mathbb{E}_t [\log \pi_\theta(a_t|s_t) \cdot G_t] \quad (2.17)$$

The actor aims to maximize the expected return of policy π_θ .

2.3.3 Proximal Policy Optimization

The Proximal Policy Optimization (PPO) algorithm follows an actor-critic scheme and introduces a novel objective function to improve upon standard policy gradient methods: the *clipped surrogate objective function*. It manages to capture some of the advantages associated with other policy gradient algorithms previously introduced, such as Advantage Actor Critic (A2C) (Mnih et al. (2016)) or Trust Region Policy Optimization (TRPO) (Schulman et al. (2017a)), yet offering a simpler and more general implementation.

PPO takes sequential optimisation steps to update the policy. Let $r_t(\theta)$ denote the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (2.18)$$

where π_θ represents the updated policy with the proposed optimisation step and $\pi_{\theta_{old}}$ before taking that step. If $r_t(\theta) > 1$, the action a_t at state s_t is more likely under the new policy than the old policy; if $0 < r_t(\theta) < 1$, the opposite. Therefore, this probability ratio

is a good estimator of the divergence between old and new policy. This ratio is used in the “surrogate” objective used by TRPO, defined as

$$L^{CPI}(\theta) = \widehat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \right] = \widehat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right] \quad (2.19)$$

and first proposed by (Kakade and Langford (2002)). The term $\hat{A}_t$ is known as the *advantage function* and it represents the relative value of an action in a given state as compared to the average action in that state. In simpler terms, the advantage function quantifies how much better (or worse) an action is for the current state, relative to the average action. The intuition behind this objective is to adjust the policy based on the advantage of actions so that, if an action has a positive advantage, the likelihood of that action under the new policy is increased, and the opposite for actions with a negative advantage. We weigh those updates with the ratio r_t , instead of the log-likelihood of actions we can find in the standard policy gradients objective L^{PG} .

We also define the ‘clip’ function as follows:

$$\text{clip}(x, a, b) = \max(\min(x, b), a), \quad \text{where } a < b \quad (2.20)$$

which ensures that the value x stays in the range $[a, b]$.

Both the L^{CPI} objective and the clip function are key components of what is then called PPO’s *clipped surrogate objective*, defined as follows:

$$L^{CLIP}(\theta) = \widehat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.21)$$

This takes the minimum between the clipped and unclipped objective L^{CPI} . Without any constraints, if the action taken is much more probable in the new policy, we might end up with an excessive update. Therefore, the clipping function is used to limit the policy update by restraining the ratio $r_t(\theta)$ in the range $[1 - \epsilon, 1 + \epsilon]$, where ϵ is a hyperparameter that controls the magnitude of the update. Typically $\epsilon = 0.2$ is chosen and thus r_t restricted to $[0.8, 1.2]$.

Effectively, this causes the gradients to be zero when the ratio is clipped, as $\nabla_\theta(1 - \epsilon)\hat{A}_t = 0$, $\nabla_\theta(1 + \epsilon)\hat{A}_t = 0$. That is, if the ratio r_t is outside the $[1 - \epsilon, 1 + \epsilon]$ range, we do not update our policy at all.

A common choice of advantage estimator, first introduced by (Kakade and Langford (2002)), is the following

$$\hat{A}_t = -V(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T) \quad (2.22)$$

where t specifies the time index in $[0, T]$, within a given length- T trajectory segment.

Algorithm 1 Proximal Policy Optimization (PPO)

- 1: Initialize policy parameters $\theta = \theta_0$ and value function parameters $\phi = \phi_0$
- 2: **for** $k = 0, 1, 2, \dots$, **do**
- 3: Collect a set of trajectories $\mathcal{D}_k$ by executing the old policy π_{θ_k} in the environment
- 4: Compute rewards-to-go $\hat{R}_t$
- 5: Compute advantage estimates $\hat{A}_t$ based on trajectories $\mathcal{D}_k$ and current value function v_{ϕ_k}
- 6: Update the policy by optimising the PPO-CLIP objective (plus entropy bonus):

$$L^{CLIP+ent} = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] + c_{ent} S[\theta](s_t)$$

where $S[\theta](s_t)$ is the entropy of θ in state s_t and $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$.

- 7: Update the value function by minimising L^{VF} to the returned values $\hat{R}_t$ via gradient descent:

$$L^{VF}(\phi) = \mathbb{E}_t \left[(v_{\phi}(s_t) - \hat{R}_t)^2 \right]$$

- 8: **end for**
-

The PPO algorithm follows an actor-critic scheme. Each iteration begins with the collection of trajectories by executing the current policy in the environment, followed by advantage estimation as previously described. Then, the policy (*actor*) is updated by maximising the PPO’s clipped surrogate objective (L^{CLIP}), adding an entropy bonus to further encourage exploration. Simultaneously, the value function (*critic*) loss is optimised (L^{VF}), to minimise the difference between the current value estimates and the observed returns. The pseudocode for this process is displayed in Algorithm 1¹.

2.4 The role of rewards

Rewards play a crucial role in reinforcement learning, serving as the primary guiding signal that drives the learning process of an agent. Policy gradient methods are an example of this, but this applies to any reinforcement learning algorithm. The paradigm is based on maximising the expected cumulative rewards, and thus steering the agent’s behaviour in that direction. Ng et al. (1999) summarises this by stating that the task is defined by the reward function, and furthermore, given the model and reward function, the optimal policy is determined.

¹PPO-CLIP algorithm (OpenAI (2022))

2.4.1 Intrinsic motivation

We can classify rewards into *intrinsic* and *extrinsic*, despite not being a clear consensus about how to exactly define each category and where to draw the line. Typically, extrinsic rewards are directly linked to the completion of the agent’s goal and are closely related to the environment. On the other hand, intrinsic rewards are not directly associated with task completion but serve as motivation to explore and learn the environment and thus may indirectly contribute to beneficial behaviours.

An example of this can be illustrated with a maze. In this scenario, an extrinsic reward is given to the agent when it finds the exit, accomplishing its task. On the other hand, intrinsic rewards may be assigned for exploring unvisited areas within the maze. While exploring new areas does not directly contribute to the primary goal of finding the exit, this ”curiosity-driven behaviour can help the agent in completing its task.

As with many other concepts in reinforcement learning, the concept of intrinsic rewards bears a similarity to human curiosity. Much like the agent in the maze, intrinsic motivation in humans leads to knowledge and skills that might result in a better understanding of the environment and beneficial behaviours in the future.

2.4.2 Reward shaping

In order to address the challenge of reward design, a common approach is reward shaping. This technique involves modifying the original reward function by adding supplementary, intermediate rewards that guide the agent towards better trajectories. These modifications use domain knowledge of the environment to reward actions that we know will be beneficial. A simple pattern of rewards can make the difference between efficient learning and an intractable problem. This heuristic is especially useful in model-free reinforcement learning, the subset of approaches that, unlike model-based, use no information about the transition dynamics of the environment to guide the learning process. A weakness of these approaches is that they may spend a lot of time in the first stages of learning, as learning only starts when the first reward is given. In scenarios where rewards are sparse, this is particularly problematic.

A good example is again the maze problem. In the early stages, the agent might spend a lot of time until it finds the exit to the maze for the first time, an event that can be rare for a random policy (Abel et al., 2021). Instead of only giving a reward when the goal is reached, we can shape the reward by giving small positive rewards for actions that move the agent closer to the goal. This would guide the agent towards discovering the correct path faster, immensely increasing learning efficiency. However, this would require knowing where the exit of the maze is, and domain knowledge of the environment is not always accessible.

Formally, reward shaping can be defined as follows (Ng et al. (1999)). We consider the problem of learning an optimal policy on an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$. However, to make the learning process faster, we introduce a transformed MDP $\mathcal{M}' = (\mathcal{S}, \mathcal{A}, p, r', \gamma)$. In this alternative MDP, r' is a transformed reward function of the form

$$r'(s, a) = r(s, a) + F(s, a) \quad (2.23)$$

where F is a bounded real-valued function $F : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, called the *reward shaping function*. The main idea of reward shaping is that, although our end goal is to learn the optimal policy for $\mathcal{M}$, we train our learning algorithm by running it over the transformed MDP $\mathcal{M}'$, with the hope of guiding the learning algorithm more effectively. By augmenting the original rewards, we provide additional feedback to the agent, potentially making the learning landscape smoother and more navigable.

Nonetheless, this assumes that learning a policy for $\mathcal{M}'$ is the same as for $\mathcal{M}$. This might not be always true, as some transformations of the reward function might completely change the task at hand. (Randløv and Alstrøm (1998)) described how some reward shaping can lead to undesired behaviours. In the maze example, if we add positive rewards when the agent gets closer to the goal but no penalty when the agent moves away from it, the agent can learn to move back and forth towards and away from the goal quickly to obtain the positive rewards without making any real progress. Ng et al. (1999) studied the conditions under which this is true and established specific conditions for reward shaping to be policy invariant. That is, the optimal policy for $\mathcal{M}$ remains optimal for $\mathcal{M}'$ even after the reward transformation. Their work proved that if the shaping function F is a *difference of potentials* at consecutive states, the policies remain invariant. That is, the reward shaping function F can be written as

$$F(s, a) = \gamma\Phi(s') - \Phi(s) \quad (2.24)$$

where Φ is a function over the states, called potential and s' is the next state after taking action a in state s . According to their results, this form prevents the introduction of artificial optimal policies, thereby ensuring that the guidance effect of reward shaping does not affect the optimality of the resultant policy.

Chapter 3

Related work

Policy gradient methods Policy gradient methods parameterise and learn the policy itself, mapping states to actions directly. The foundational theorem of policy gradients was first established by Sutton et al. (2000), introducing the idea of optimising the policy via gradient ascent and proposing the REINFORCE algorithm as an implementation of this concept. However, vanilla implementations of the policy gradients method often suffer from high variance, leading to unstable learning.

In order to address these issues, several policy gradient algorithms were later proposed following the actor-critic framework, which uses two networks: the actor and the critic. The actor learns the policy and decides the actions to take, while the critic learns to estimate the value of the chosen actions and provides a learning signal that the actor can optimise. One of the early contributions to this was the Asynchronous Advantage Actor-Critic (A3C) algorithm (Mnih et al. (2016)). Following this, Soft Actor-Critic (SAC) (Haarnoja et al. (2018)) integrated entropy regularisation into the objective function to encourage exploration; Trust Region Policy Optimization (TRPO) (Schulman et al. (2017a)) ensures the new policy does not deviate too much from the old one at each step; and Proximal Policy Optimization (PPO) (Schulman et al. (2017b)), enforces the TRPO constraint via a novel objective function, called *clipped surrogate objective*, making it simpler to implement. Further techniques, namely, the aforementioned Experience Replay and Generalised Advantage Estimation (GAE) (Schulman et al. (2018)) were also added to policy gradient methods to provide more stable and efficient learning. In recent years, policy gradient algorithms have become standard benchmarks in deep RL due to their robust performance and ability to handle high-dimensional, continuous action spaces.

Reward design Given the importance of rewards to the learning process, the choice of reward function can determine the success of an RL agent. The notion of an *optimal reward function* is considered by Barto et al. (2009), defined in simple terms as the function that

helps the agent learn the optimal behaviour for the given task as efficiently as possible. As pointed out by that study, the optimal reward function may not always directly relate to the completion of the task. Hadfield-Menell et al. (2020) describe the complexity of designing a reward function that actually captures what we want and thus leads to the desired behaviour. This makes reward design a complex and nuanced task in reinforcement learning. The concepts of *extrinsic* and *intrinsic* rewards come into play in this context. Typically, we define extrinsic rewards as those directly associated with task completion, whereas intrinsic rewards are the ones which encourage exploration or environment learning and thus only indirectly support the task at hand. Just like other notions in reinforcement learning, we can relate intrinsic motivation to the psychological notion of curiosity, and similarly to psychology, the line between extrinsic and intrinsic reward is not clear. As suggested by Singh et al. (2010), there are no hard features that distinguish extrinsic and intrinsic motivation, and instead, these relationships might vary along a *continuum*.

Reward shaping Reward shaping is a technique that exploits the interplay between intrinsic and extrinsic rewards. This practice modifies the reward function to enhance the learning process, typically by incorporating intermediate rewards that encourage beneficial behaviours and guide the agent towards the desired goal. Some early studies started using reward shaping in reinforcement learning applications (Randløv and Alstrøm (1998); Colombetti et al. (1996)). One of the big steps into the formalisation and deeper understanding of the subject was Ng et al. (1999), which introduced Potential-Based Reward Shaping (PBRS) and policy invariance principles, demonstrating that properly designed shaping can enhance learning without altering the optimal policy. Some works in large-scale applications have used handcrafted reward shaping by making use of extensive prior knowledge (OpenAI et al. (2019); Yu et al. (2021)). The requirement of prior knowledge is, precisely, one of the issues we would like to tackle in our work. A possibility in that direction is to use uninformed exploration methods such as intrinsic rewards or state-count-based algorithms (Raileanu and Rocktäschel (2020); Bellemare et al. (2016)). In our study, we attempt to use a completely domain-agnostic variable such as time to shape the reward and guide the agent’s learning process.

Survival as a goal in reinforcement learning Several previous works have explored the possibility of modelling agent survival and using this to learn. One example is Yoshida (2016), where the reward function is expressed as the temporal survival probability and it is empirically proved that the agent then learns survival behaviours. Moazami and Doerschuk (2020) approach the problem with a model-based approach introducing a novel method that focuses on avoiding terminal states rather than maximizing accumulated rewards from safe states in order to maximise agent survival time. Ringstrom (2022) used the idea of using

survival time to drive learning further and proposed a self-preserving agent that does not use reward-maximization as an objective. However, this approach involves defining a new formalisation called Temporal Goal Markov Decision Process (TGMDP) and a new class of Bellman equations, which differs considerably from the standard RL paradigm. The aforementioned studies suggest that survival time could serve as an effective reward function, and the main focus of this work will be to fully understand the viability and implications of this approach in various RL problems.

Chapter 4

Soft approach: Reward Shaping with Time

This chapter focuses on the first proposed approach in our study, namely the *soft approach*. This strategy involves shaping the reward by adding a cost per timestep, which in episodic settings relates to the length of the agent’s lifetime. While being a simple approach that does not modify the MDP classical structure, studying the effects of these changes on the reward function can provide a perspective on how small modifications that do not use any task-specific knowledge impact the learning process.

In this chapter, we first present the approach from a conceptual, environment-agnostic point of view. The second section details the experiments conducted within the framework of this approach, describing the goals and expected findings of the trials. This is followed by a presentation of the results, displayed by various plots and graphs. Finally, this chapter ends with a discussion of the approach and any patterns, trends and significant insights derived from the experimental outcomes.

4.1 Methodology

The soft approach is built around the concept of reward shaping, which, as described previously, aims to guide an agent towards certain desired behaviours by modifying its reward function. The modifications included should not change the end goal of the agent but might include rewards for subgoals implicitly related to the main task. While reward shaping can be very useful to learn certain behaviours that the agent would not learn with standard reward design or to drastically increase the learning speed, it requires task-specific knowledge and understanding of the dynamics of the environment. What we first propose is using the simplest way of shaping rewards: using the lifetime of the agent.

For this approach, we consider environments consisting of reaching a certain goal or

position in a fixed amount of time. A reward-shaping strategy that adds a cost to time is then expected to encourage the agent to find the most time-efficient behaviour to complete the task. While this is a general simple practice in reward design, we explore it as a general strategy that could improve performance in any of the described environments without the need for even knowing the environment when adding the reward shaping.

The proposed method consists of including a cost of time into the original reward. Formally, given an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$, we run our learning algorithm on a modified $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r', \gamma)$. Here r' is the modified reward function $r'(s, a)$, defined as:

$$r'(s, a) = r(s, a) - C(s) \quad (4.1)$$

$C(s)$ is a positive function that adds a cost to time and acts as the reward shaping function $F(s, a)$ previously defined (2.23). The introduction of the time cost $C(s)$ into the reward function acts as a penalty for each time step the agent takes. The simplest setting is to choose a fixed time cost $C(s) = c$ with $c > 0$, which uses no information about the state of the agent or about how far it is from completing the goal. Given that C only depends on s , it falls under the potential-based shaping function defined in Eq. 2.24 and thus ensures that the optimal policy in $\mathcal{M}'$ is the same as in $\mathcal{M}$. This means that maximising the cumulative return with time cost is equivalent to doing it in the original reward function and thus both agents are completing the same task.

We call this approach ‘soft’ because it retains the standard MDP framework used in conventional reinforcement learning, making adjustments only to the reward design. Therefore, the learning dynamics under this approach are expected to be similar to those under the original reward, but with greater potential for task-solving efficiency and learning speed in a wide range of environments.

Despite strategies that add a time penalty to encourage agents to solve fast in the minimum amount of steps have been used in reinforcement learning, but none of these studies explicitly and thoroughly investigated its effects. We aim to use it as a broadly-applicable method across diverse environments, without the need for intricate task-specific knowledge. However, we consider this approach not as the main proposed methodology, but rather as an intermediary stage which explores the effects of a time penalty in goal-rewarded tasks, before moving to a survival-time dependent, ‘rewardless’ system.

The central question we aim to answer is: *Can the addition of a cost on time improve the agent’s performance in goal-oriented tasks? How?* To answer this question, we use Experiment I, whose main goal is to investigate the effects of adding a time cost to the reward function. In the following sections, we describe the design and results of Experiment I, followed by a discussion of the findings.

4.2 Experiment I

4.2.1 Experimental setup

MiniGrid (Chevalier-Boisvert et al. (2023)) is a reinforcement learning environment suite with grid worlds implemented on Python that serves as a simple testing ground for RL agents. It includes a variety of environments with different characteristics and tasks, ranging from simple navigation to harder exploration and memory problems.

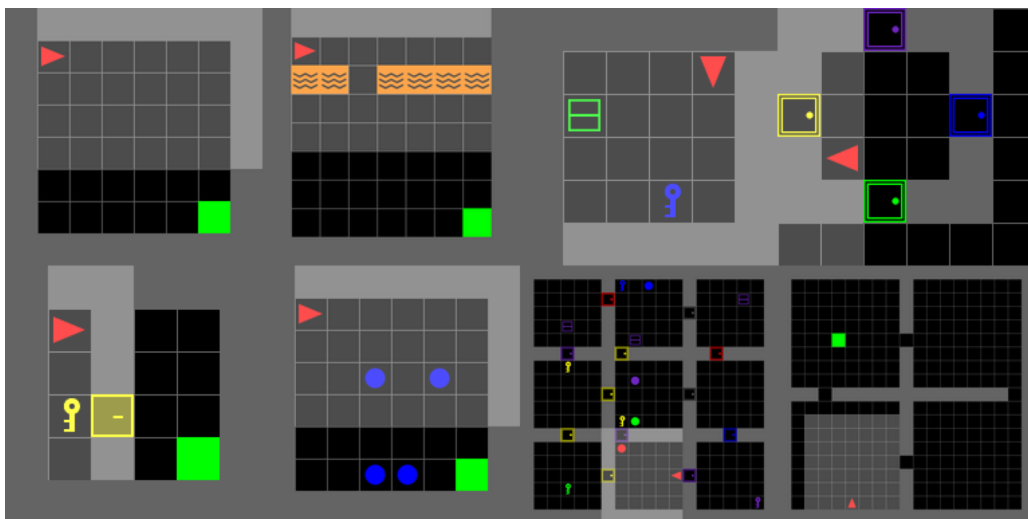


Figure 4.1: Render of 8 different MiniGrid environments

We use the partially-observable symbolic representation observation of shape $(7, 7, 3)$. This provides the agent with a 7×7 field of view representing its immediate surroundings from its perspective, where each cell is represented with a 3-tuple $(\text{obj_id}, \text{colour}, \text{state})$. The action space consists of 7 actions: **turn left**, **turn right**, **forward**, **pick up**, **drop** an object, **toggle** and **noop**. These actions allow the agent to interact with the environment and complete the tasks. For environments where any of the actions does not apply (e.g. **pickup** for environments without objects), that action simply has no effect.

Agent architecture The chosen agent architecture consists of a CNN base comprising two convolutional layers with max pooling, which uses shared weights for both the actor and the critic. The output of this network is then flattened and passed through separate fully connected layers for the actor and critic heads. The actor head outputs a probability distribution over the possible actions, from which we then sample to select actions. The critic head outputs a value estimate for the current state, which is used to calculate the advantage in the policy gradient update. This architecture is depicted in Figure 4.2.

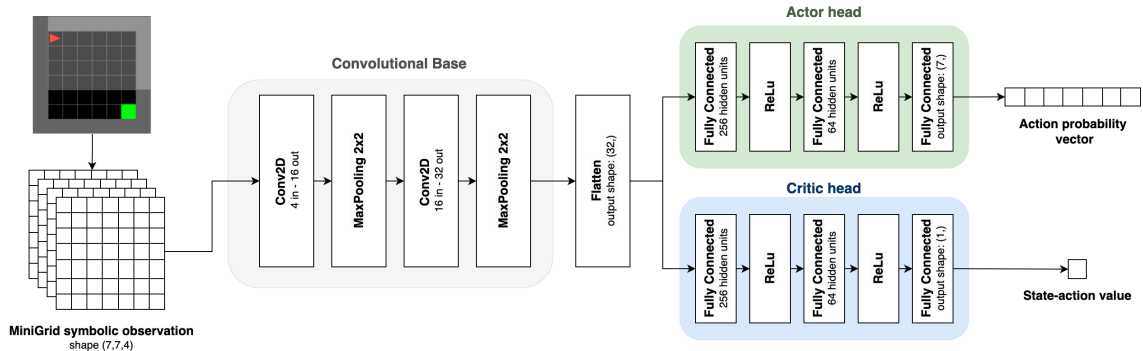


Figure 4.2: Agent’s architecture diagram

PPO hyperparameters The values for PPO hyperparameters were picked following common values in literature, adjusting some of them for our specific environments (e.g. number of timesteps per policy rollout). The final set of values used in the experiments is shown in Table 4.1.

Table 4.1: PPO hyperparameters

hyperparameter	value	description
learning-rate	2e-5	learning rate for the policy optimization algorithm
num-envs	32	number of environments running in parallel
num-steps	256 ¹	number of timesteps per policy rollout (per environment)
gamma	0.99	discount factor γ
gae-lambda	0.95	λ parameter in the GAE
num-minibatches	16	number of mini-batches used in the policy update
update-epochs	4	number of epochs during the policy update
clip-coef	0.2	clipping coefficient for the PPO-CLIP objective
ent-coef	0.01 ²	coefficient for entropy regularization
vf-coef	0.5	coefficient for the value function loss in the combined loss

4.2.2 Experiment design

Environments In this experiment, we focus on navigation goal-oriented problems, in particular, environments from the MiniGrid suite, where the agent’s objective is to reach a fixed goal position where the episode terminates. For all of our experiments, we use a partial observation where the agent receives a frontal view of size 7×7 .

- **MiniGrid-Empty-16x16:** A simple and straightforward environment consisting of an open 16×16 grid, where the agent starts in one of the corners and needs to reach the goal in the opposite one. Start and goal positions are fixed but the reward signal is sparse.

- **MiniGrid-SimpleCrossingS9N2:** This environment consists of a 9x9 grid with two walls with gaps, creating a 'maze' the agent needs to navigate to reach the goal.
- **MiniGrid-FourRooms:** This environment consists of a large grid with four rooms interconnected by 4 gaps in the walls. Both the agent and the goal are randomly placed in any of the four rooms, which might make learning a challenging task which requires multi-room navigation and exploration.
- **MiniGrid-Unlock:** In this environment, the agent must find a key placed in the grid to unlock a door to finally reach the goal. It requires sequential planning of actions to solve the task.
- **MiniGrid-DoorKey-8x8:** It contains a locked door blocking the path to the goal, and the agent must first find a key located somewhere else in the grid to unlock the door. This task poses more complexity and requires exploration to find the right sequence of actions needed to reach the goal.
- **MiniGrid-KeyCorridorS3R2:** The agent must find a key located behind some doors to unlock the final doors where the object which the agent needs to pickup is located. This task requires doing the sequence of actions in the correct order.

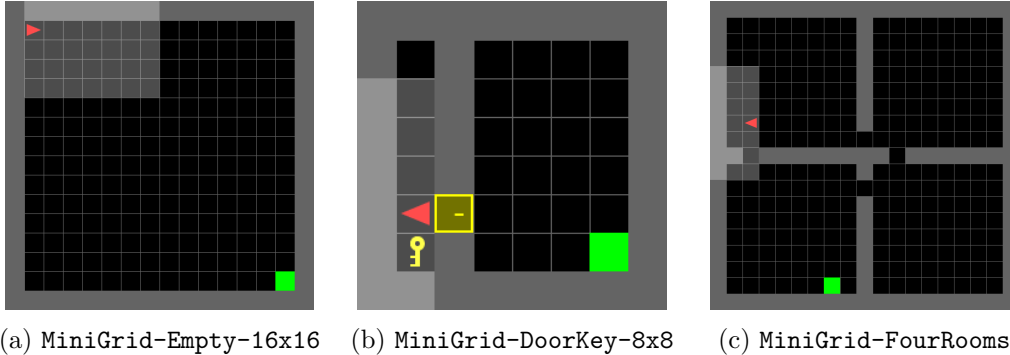


Figure 4.3: Environments used in Experiment I

A graphical representation of the three described environments is shown in Figure 4.3.

Procedure We train agents with PPO-CLIP under three different reward settings, aiming to evaluate the impact of different time cost configurations on the agent’s learning process.

The general reward function can be defined as:

$$r(s, a) = G \cdot \mathbb{I}[\text{goal reached in state } s]^3 - C \quad (4.2)$$

³ $\mathbb{I}[\text{statement}]$ is the indicator function which takes value 1 if statement is true and 0 otherwise

where G is the reward for reaching the goal and C denotes the cost incurred at each timestep.

We compare results for three distinct reward function settings for agents: A, B and C. Out of these, Agents A and B act as baselines, illustrating conventional reward structures, in which the agent only receives the reward when reaching the goal. In contrast, the proposed agent Agent C introduces the time-cost shaping at every timestep. The reward functions for each of the three settings are defined as follows:

- **Agent A** (standard 1/0): $G = 1$, $C = 0$. The agent receives +1 for reaching the goal, 0 otherwise.
- **Agent B** (final penalty): $G = 1 - P$, $C = 0$. Agent receives $+(1 - P)$ for reaching the goal, 0 otherwise. We use the default MiniGrid penalty

$$P = 0.9 \cdot \frac{\text{timesteps}}{\text{max_timesteps}}$$

so that $r(s, a) \in [0.1, 1]$.

- **Agent C** (time cost): $G = 1$, $C = \text{timecost}$. The agent gets +1 for reaching the goal but incurs a cost of $-(\text{timecost})$ at every other timestep. For the experiments, we set

$$C = 0.9 \cdot \frac{1}{\text{max_timesteps}}$$

so that the episodic return of agents B and C is equivalent when they take the same amount of steps.

Metrics We use the following metrics to compare the performance of agents.

- **Success rate:** proportion of runs where the agent reaches the goal (between 0 and 1).
- **Episode return.** It might not be directly comparable with different reward designs.
- **Episode length** (in timesteps). Since episodes terminate only when the goal is reached or the episode is truncated, a shorter length will be an indicator of better performance.

4.3 Results and discussion

In an effort to reduce the noise and alleviate the effect of outliers in order to improve the robustness and reproducibility of these results, multiple runs (between 20 and 50) were

used to train each agent configuration. The graphs and figures presented throughout the chapter summarise the aggregated runs, displaying the median (represented by lines) and standard deviation (shaded areas), unless otherwise indicated. This choice follows the observation that, for our experiments, the median captures more accurately the shape and differences between agents than the mean.

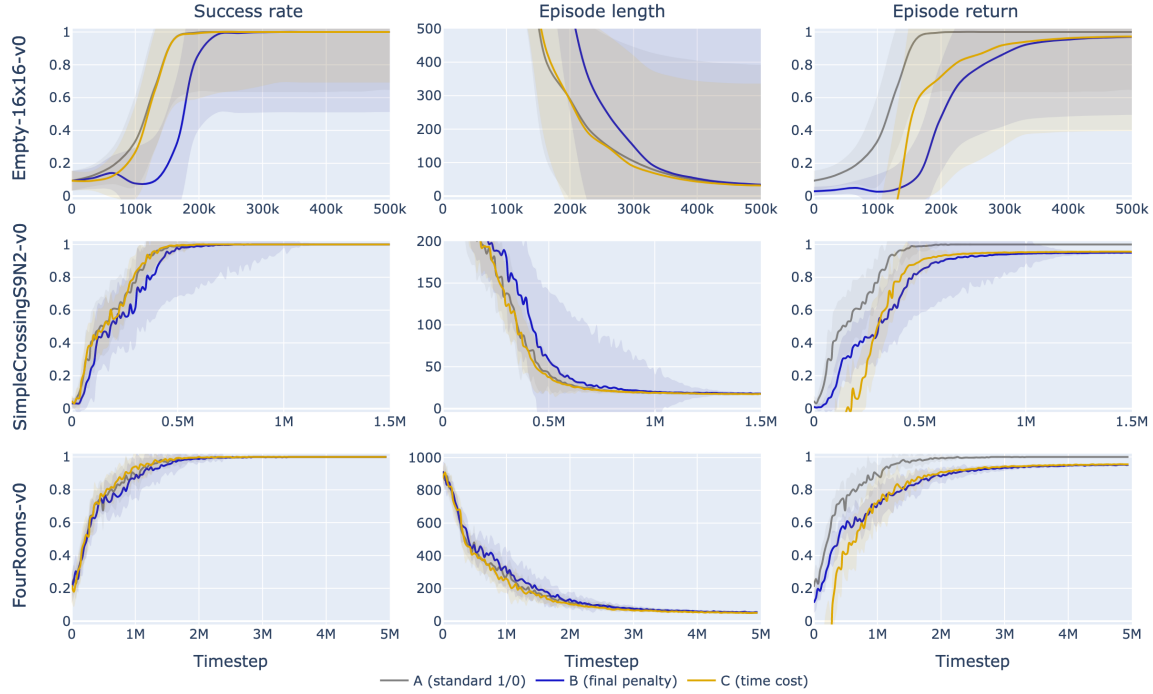


Figure 4.4: Experiment I results on the following environments: MiniGrid-Empty-16x16, MiniGrid-SimpleCrossingS9N2, MiniGrid-FourRooms

Figure 4.4 presents the training results of Experiment I on three MiniGrid navigation environments where the agent needs to reach a goal with different levels of complexity and sparsity. These plots depict the evolution of performance along the training using PPO.

Given that for these environments the goal is to reach the goal as quickly as possible, we include episode length along with success rate and episode return as metrics to follow. Note that while success rate and episode length are directly comparable, episode return will be higher in the case of agent A, which gives +1 for reaching the goal and includes no penalty for the number of steps. On the other hand, agents B and C cannot obtain an episodic return of 1 since a negative penalty is always included.

Results on navigation environments The first observation is that all three agents solve the task and converge to the same policy, which in every case is very close to optimal. This is consistent with our intention to design three reward functions that encode the

same task and preserve policy optimality. However, the learning efficiency and stability seem considerably different. The performances of agents A and C are closely aligned and outperform B. Both learning curves are very similar and there is no significant difference between them. On the other hand, agent B learns slower, taking more steps to reach convergence. Moreover, it is less stable; standard deviation shaded areas indicate that there are more runs that diverge or deviate, taking considerably more steps than the majority. This might indicate that the learning signal for B is unstable whereas A and C are clearer for the learning algorithm.

An interesting result is that A, without a step penalty designed to encourage the agent to reach the goal as fast as possible, reaches the goal in the same number of steps as C on average. Moreover, the PPO training of A until convergence is as fast as C, making the learning process of both agents nearly identical. This result is unexpected; we would expect A to learn paths that reach the goal in a number of steps that might not necessarily be optimal.

One possible explanation for this effect is that PPO already directs the agent into the states yielding a higher value and those which are the closest to the goal, effectively driving the agent to the goal through the optimal path. Another possibility is that the reward function also impacts the exploration-exploitation tradeoff. Since reward function A adds no penalty for taking additional steps, the agent might take more "exploratory steps", allowing the agent to discover the optimal path faster. This might offset any advantage that the time penalty could give to Agent C.

Results on more complex environments In order to further confirm these trends, we also trained the agents on three other MiniGrid environments requiring more complex exploration. Instead of just navigating through the map and reaching a position to complete the task, these environments also require picking up objects, opening boxes and unlocking doors, making these tasks more challenging. These are presented in Figure 4.5. In these three new environments, we can again observe the same trends, further supporting the insights previously outlined.

A second interesting observation is that B performs considerably worse than the others, in particular, worse than C. By design, B and C are very similar in the sense that, for a trajectory taking the same number of steps, the total episodic return is the same. Recall that this happens due to the design of the reward functions: B includes a step penalty at the end, whereas C, the *time cost* is uniformly spread over every timestep of the episode. Therefore, we could expect B and C to perform very similarly, but this is not the case.

This phenomenon might be explained due to the fact that PPO relies on state value estimation by the critic. In this context, the reward function used by B gives noisy and conflicting signals, especially in the first stages of the learning process when the agent

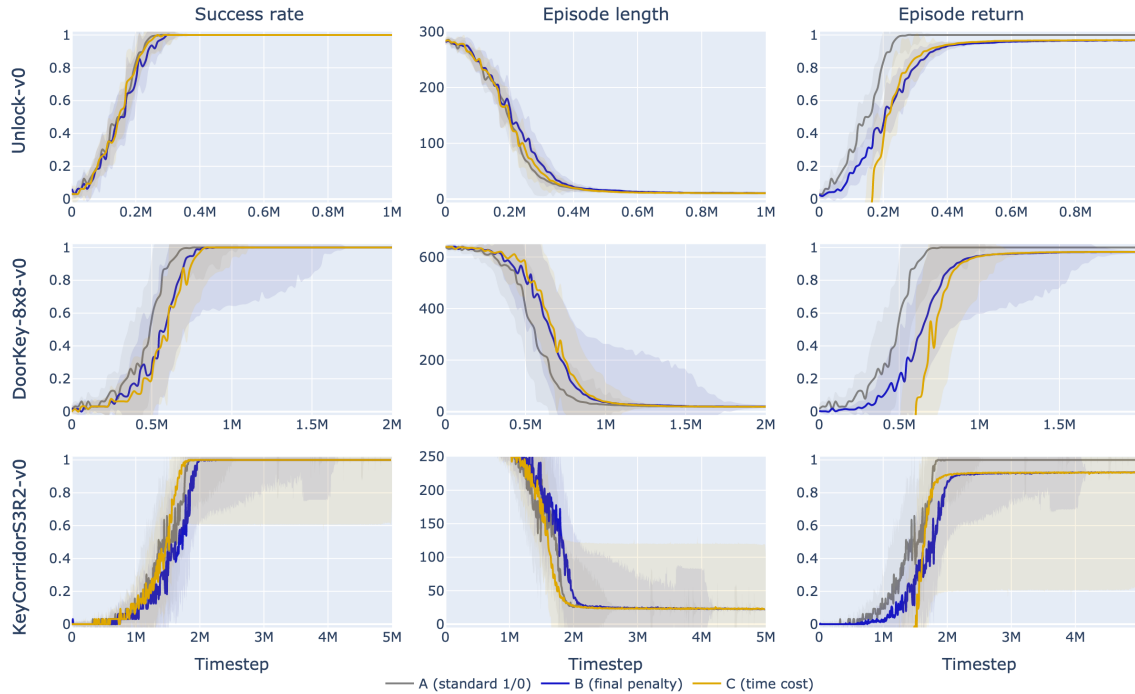


Figure 4.5: Experiment I results on the following environments: MiniGrid-Unlock, MiniGrid-DoorKey-8x8, MiniGrid-KeyCorridorS3R2

might reach the goal "by chance" within varying amounts of steps. This inconsistency might introduce a greater complexity in learning a value estimation for the states, leading to slower and less stable learning. Conversely, A and C might offer clearer signals that contribute to better performance.

Nonetheless, this is only true when using bootstrapping. For the previous experiments, we use Generalised Advantage estimation (GAE) $\lambda = 0.95$. Recall that $\lambda = 1$ is Monte Carlo, where the return is computed using complete episodes, and $\lambda = 0$ is purely TD learning, where we only the next step is used. Intermediate values like the one used mean that we consider a combination of these two methods, weighing future rewards with decaying importance. We would expect that with $\lambda = 1$ agents B and C should perform very similarly since the episodic return is the same with both reward configurations for the same number of timesteps, and the differences to only be visible when using bootstrapping.

Effect of bootstrapping In order to explore this hypothesis, we compare the agents with different values of λ_{gae} , presented in Figure 4.6. The plots show that the performance is worse as λ decreases. More interestingly, the differences between agents B and C are inexistent for $\lambda = 1$ and appear in the other plots, where we use bootstrapping. This might indicate that the value of λ has a substantial influence on the performance differential

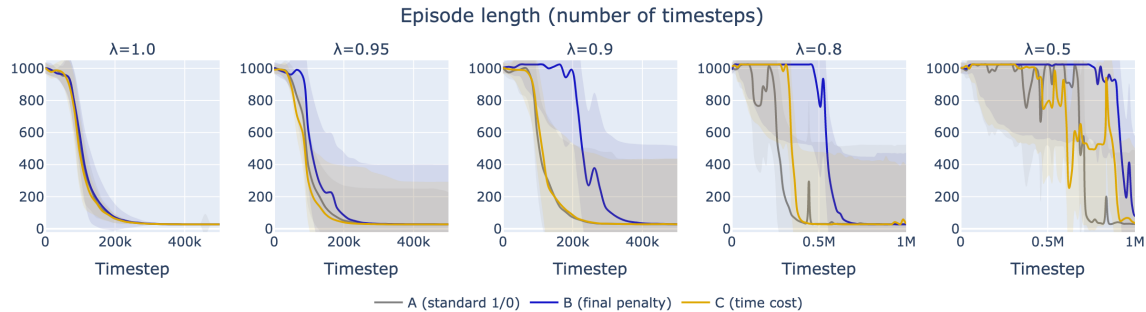


Figure 4.6: Experiment I results on the **EnergyBoxes-Empty-16x16** environment for different value of the GAE λ parameter. Note that the x-axis, representing the number of steps in the training process, is different for each plot, given the varying training speeds.

between the reward configurations and that A and C are better than B, especially in the context of bootstrapping.

Summary and implications In summary, the results show how the proposed reward with time cost (agent C) has more stable and faster learning than the default MiniGrid agent with the steps penalty at the end (agent B). These effects are consistent across several environments, suggesting that the soft approach can be an effective and simple way of encoding the cost of time. However, Experiment I is also an example that reward design is a nuanced and complex aspect of reinforcement learning, and some results might be unexpected and counterintuitive. In this experiment, we found that agent A, without a specific penalty designed to minimize steps, learns optimal paths to the goal as efficiently as agent C with the time cost, with A consistently matching the optimal performance in all environments. This result, though surprising, aligns with other examples in literature such as Hadfield-Menell et al. (2020) describing how hard it is to design rewards that accurately capture what we want.

The outcomes from the soft approach are promising, as they reveal that time can indeed be used as an enhancing element for reward shaping. However, these effects suggest a bigger potential to take a step further and use time as sole reward: the *hard* approach. This leads us to the next chapter in our study where this idea will be explored.

Chapter 5

Hard Approach: Lifetime as Reward

This chapter covers the second proposed approach of our study, the *hard approach*. In contrast to the soft approach, this represents a more drastic strategy where the agent’s learning is driven by its lifetime as the sole explicit reward. It removes traditional rewards for achieving goals and embeds this information into the environment indirectly via termination probabilities, simulating real-world scenarios where the survival dynamics might not be explicitly given to the agent, but still define the environment.

The chapter follows the same structure as the soft approach. We first introduce the methodology and describe the experimental design and setup. Then, we present the obtained results and finish with a general discussion that extracts conclusions about the viability and applicability of the approach.

5.1 Methodology

The proposed *hard* approach can be summarised as “Lifetime as the only Reward”. The lifetime of the agent is the only form of reward the agent receives, removing any explicit reward for reaching goals and putting all the emphasis on survival time. This is a more drastic perspective than the one used for the soft approach and leaves more room for different behaviours and learning dynamics.

A key component of this approach is what we call the *energy budget*. This concept refers to environments where the agent’s lifetime depends on a finite amount of energy, which depletes over time. When this energy reserve drops to zero, the episode ends. This energy budget is not known to the agent, the only information the agent receives is the episode termination. This can be true for many environments in real life where doing some actions that are not directly rewarded may extend survival time and thus allow for higher

potential rewards.

In these environments, doing certain actions or reaching goals replenishes the energy budget. The key idea here is that we substitute the goals that would give positive rewards to the agent with energy replenishment, thereby embedding the goal-oriented behaviour into the system without the need for explicit rewards. That is, the energy budget and dynamics are assumed to be part of the MDP, instead of encoded by the reward function.

More formally, let e be the energy budget which is initialised to an initial energy value e_0 . We also define the energy replenishment function $ER(s, a)$ which defines the states-action pairs (s, a) which refill the energy budget. The termination of an episode occurs when the energy budget drops to 0. These dynamics are depicted in 2, which describe how the energy budget environments work. The pseudocode omits the details of the learning process, as this can be integrated seamlessly with any existing reinforcement learning algorithm.

Algorithm 2 *Energy Budget* style environment

```

1: Initialize state  $s$  and energy budget  $e \leftarrow e_0$  ▷ Set up initial energy  $e_0$ 
2: Episode starts
3: while  $e > 0$  do
4:   Choose action  $a$  from state  $s$  using current agent's policy  $\pi$ 
5:    $e \leftarrow e - 1$  ▷ Energy decreases with each step
6:   Compute next state  $s'$  and receive reward  $r(s, a)$ 
7:    $e \leftarrow e + ER(s', a)$  ▷ Energy replenishment for reaching goal
8:    $s = s'$ 
9: end while
10: Episode termination

```

One could argue that even though goal information is no longer provided directly in the form of rewards, such objectives still need to be manually encoded within the energy dynamics. However, our rationale is that in many real-world, scenarios these survival dynamics are inherent. An example of this could be a game like *Minecraft*, where the player's goal could be to explore and collect as many objects as possible while surviving. The player's survival depends on health management and avoiding enemies and hazards. Although there are other tasks like exploring and building, simply rewarding survival time could lead the agent into learning behaviours such as safe exploration and resource gathering, effectively promoting goal-oriented behaviour that aligns with survival.

Another example in the realm of Atari games is the classic *Pac-Man*. In this game, the player moves around a maze, aiming to consume all the dots while avoiding being caught by ghosts. The main goal is to obtain as many points as possible, which are earned by eating dots and consuming ghosts. The ghost-eating ability is temporarily enabled when the player consumes "power pellets" placed in the maze. If, instead of rewarding explicitly

the agent with points, which is the ultimate objective, we only reward survival time, the agent will learn to evade ghosts and thus indirectly increase the final score. Moreover, this setup might also promote behaviours such as consuming the power pellets and chasing ghosts to consume them, as that will also extend survival time and potentially improve game performance.

In summary, in situations where survival dynamics are inherent to the environment, we can leverage a generalized reward function that remains completely agnostic to the goals and dynamics specific to the environment. Following this approach, Experiment II is designed to test this approach’s viability, exploring whether agents can learn effectively within this framework and how they compare to agents receiving explicit goal-oriented rewards. The design and results of the experiments will be discussed in the following chapters.

5.2 Experiment II

5.2.1 Experimental setup

For the second experiment, we use the same environment suite (MiniGrid), agent architecture and PPO configuration described in the previous section 4.2.1.

5.2.2 Experiment design

Objective The main goal of Experiment II is to test the viability of the hard approach. The question that we want to answer is: *Can tasks be solved in an environment where explicit rewards are absent and the only reward is the agent’s survival time?* In case the answer is yes, we also want to see how an agent using only time bonus compares to agents with the standard rewards.

Environments In this experiment, we use custom environments defined on the MiniGrid environment suite which incorporate the energy budget system. Specifically, we designed two environments, namely, **EnergyBoxes** and **EnergyBoxesHard**, inspired by one of the environments described by Barto et al. (2009). In both environments, the map is an empty grid with two *food boxes* at the corners: red box and blue box. The agent begins the episode with an initial amount of energy and must *eat* food boxes present in the environment to replenish its energy and extend its survival time. The agent can only eat from a box if it is full, otherwise, the action has no effect. After a box is eaten, it becomes empty at the next timestep. Figure 5.1 shows a graphical representation of these environments.

The specific configuration of the environments is the following.

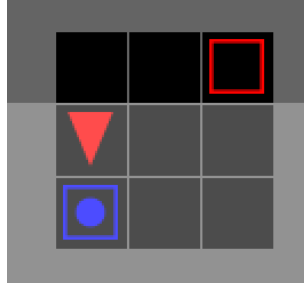


Figure 5.1: Grid of **EnergyBoxes** and **EnergyBoxesHard** environments used in Experiment II

- **EnergyBoxes**: The agent starts in one of the corners facing a random direction. Only one of the boxes is full at the start and empty boxes stochastically become full again with 0.1 probability at each timestep. The agent begins with an energy budget of 10, and eating a food box replenishes 8 units of energy. Episode truncation occurs after 256 steps.

The environment is designed so that the optimal strategy consists of moving between boxes, as the stochastic refilling of boxes guarantees a more regular food supply. Most of the times, moving between boxes will allow the agent to survive indefinitely, whereas staying in one box will result in energy depletion. However, given the stochastic nature of box refilling, the described trends are only true on average, and might not be true in some episodes.

- **EnergyBoxesDelay**: Same initial setting as **EnergyBoxes**, but here the agent begins with an energy budget of 100, and the refill probability is 0.05. The episode truncates at 512 timesteps. This configuration allows the agent to survive for a minimum of 100 timesteps, during which it might start eating boxes to increase its energy budget. This will increase its lifetime, which takes effect only after a large number of timesteps.
- **EnergyBoxesHard**: The agent starts in one of the corners facing a random direction. After one box is eaten, the other one will be refilled immediately the next timestep and will remain empty otherwise. The agent begins with an energy budget of 5, and eating a food box replenishes 6 units of energy, precisely the number of steps required to eat from one box and reach the other. The episode is truncated after 512 steps.

This environment is designed so that there is only one possible behaviour for survival: eating from one box and immediately moving to the other one in the minimum amount of steps permitted in the environment. Any deviation from this optimal behaviour results in energy depletion and thus the episode will terminate before reaching the other box. While this makes this environment very hard to learn, it also does not leave room for different behaviours that keep the agent alive.

Procedure We train two agents with PPO-CLIP under the described environments, with the following reward configuration:

$$r(s, a) = G \cdot \mathbb{I}[\text{box eaten in state } s] + B$$

where G is a parameter that controls the reward for eating a box and B is the fixed time bonus.

- **Agent A** (goal-oriented reward): $G = 1, B = 0$
- **Agent B** (time bonus): $G = 0, B = 0.1$

The purpose of these reward designs is to measure the performance of the hard approach’s survival-time reward system (Agent B), compared to a standard reward-for-goal system (Agent A), which serves as the baseline. While A is given the dynamics of survival explicitly, B is only encouraged to eat the boxes implicitly, as only survival time is rewarded.

Metrics We use the following metrics to compare the performance of agents, computed for each episode. Recall that these environments use two boxes: red box and blue box.

- **Episode return.** It might not be directly comparable with different reward designs.
- **Episode length** (in timesteps). It is an indicator of the average agent’s survival time, which we aim to maximise in this experiment.
- **Total box count:** number of food boxes of any colour eaten by the agent during an episode.
- **Consecutive box count:** number of times that the same box is eaten consecutively during an episode (e.g. eating twice the red box would count as 1 for this metric).
- **Mix rate:** measure how much the agent switches between the two boxes. Computed as

$$\text{mix rate} = 1 - \frac{\text{consecutive box count} + 1}{\text{total box count}} \quad (5.1)$$

This expression equals 0 when the agent switches every time from one box to the other one ($\text{consecutive box count} + 1 = \text{total box count}$) and equals 1 when the agent only eats from one box ($\text{consecutive box count} = 0$).

- **Agent distance.** Total length (Manhattan distance) of the path described by the agent. Rotation actions do not increase agent distance.

5.3 Results and discussion

Results of Experiment II are shown in Figure 5.2. Each row depicts the result Given the particular learning curves of the agents in these environments, we plot every individual run (with transparency) and a summary of the aggregation using the median and the mean. Looking at the curve of each individual agent, in combination with the mean and median, gives a better picture of the learning process.

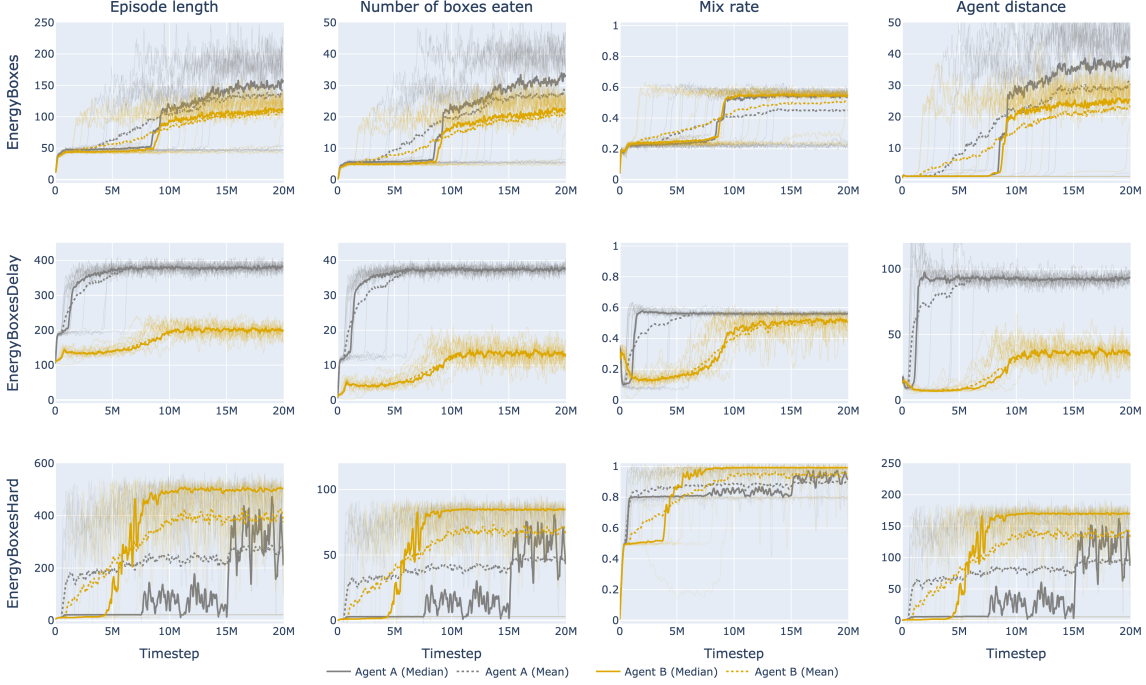


Figure 5.2: Experiment II results on the following environments: **EnergyBoxes**, **EnergyBoxesDelay**, **EnergyBoxesHard**. Each of the background lines represents one individual run, then summarized by the median and mean.

EnergyBoxes For this environment, we can observe that both agents learn to solve the task and seem to learn at a similar rate. Both agents learn a policy where the agent switches between boxes, but the final behaviour learnt by A seems to yield longer survival times than B.

Looking at each individual training run, the agents rapidly learn to go to one box and eat only from there (episode length stays around 50 and agent distance close to 0). Recall the energy configuration for this one: 10 initial energy, 8 replenishment for eating and 0.1 refill probability. This means that eating from one box and waiting will make the agent, on average, run out of energy and thus lead to episode termination. The optimal policy would be to take advantage of greater food availability by switching between boxes.

This is what happens exactly when the runs suddenly “jump” from episode lengths of around 50 to longer episodes. This clear step in performance happens to each individual run stochastically, by random exploration, and the training timestep at which it occurs varies greatly between runs. However, looking at the medians we can see that it tends to happen around the same timestep ($\sim 8M$) for both agents A and B.

However, there is still a qualitative difference in behaviour between the two agents, as we can see that, after the jump, agent A tends to achieve longer survival times than episode B. That means that, once the agent has learnt to jump between boxes, the policy that agent A learns is more effective for survival than B. This can be clearly observed by looking at the individual runs, as well as reflected on the mean and median.

While Agent A is rewarded directly for the action of eating, Agent B obtains higher return in episodes where eats more boxes, which also involves moving between boxes, going to the closest box which is full, etc. This makes the extrinsic goal-oriented reward of A a clearer signal that results in a better performance. However, it is still significant that B can achieve a performance that is very close and matches the learning speed during training.

EnergyBoxesDelay In this environment, Agent A clearly outperforms Agent B in terms of both convergence speed in the training process and performance of the final learnt behaviour.

Recall the energy configuration: 100 initial energy, 5 energy replenishment for eating and a refill probability of 0.05. Given that the energy replenishment for eating is considerably smaller than the timestep frequency at which food will reappear on average (every 20th step), the agent will eventually run out of energy before episode truncation. However, the large initial energy budget allows the agent to survive for at least 100 episodes within which it can start eating to store energy. That means that, ideally, to maximize survival time, an agent should start eating from the beginning of the episode as fast as possible.

This environment presents a learning challenge for Agent B because even if it decides to consume a box at the very first timestep, the benefits (in terms of prolonging its lifetime and thus higher return) are very delayed: it will not be effective until after the 100th timestep. This can be observed in the results, where the gap between the two agents is very big: A starts learning faster and converges in a much better policy. These difficulties can be further observed in Figure 5.3 (left), which depicts the average eat count per episode timestep of fully trained agents. We can clearly see that, while Agent A starts eating as soon as the episode starts (with a big surge at timestep 4), Agent B stays inactive in the initial phase of the episode and only starts eating closer to the 100th timestep, when the consequences of eating start immediately contributing to its survival.

While it is clear that in this case, the standard goal-oriented reward outperforms *lifetime as reward*, the bootstrapping method could also contribute to the difference. With $\lambda = 0.95$,

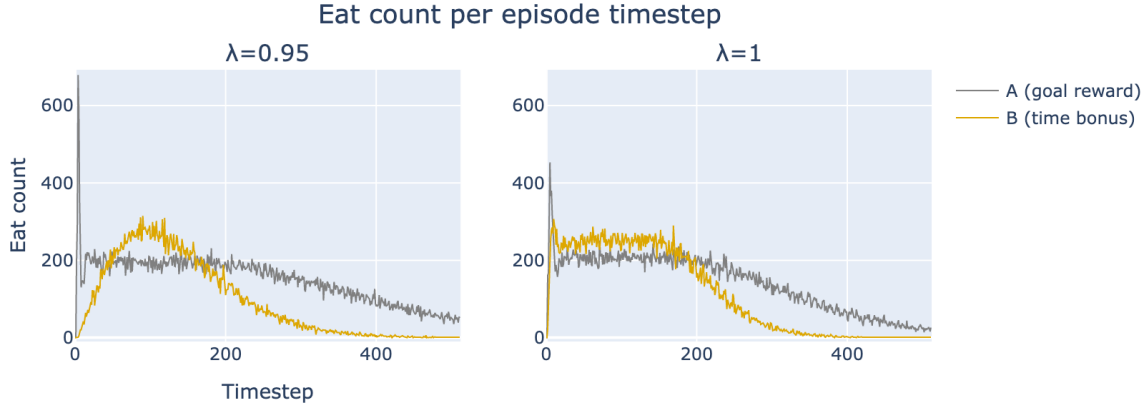


Figure 5.3: **EnergyBoxesDelay** sum of boxes eaten per episode timestep; fully trained agents (aggregated episodes 1M timesteps)

the impact of the reward decays fairly quickly for the initial steps and may not propagate sufficiently to cause changes in early episode behaviours. For that reason, we examine the effects of using full Monte Carlo returns ($\lambda = 1$). This is not the chosen configuration because, in general, some bootstrapping is generally preferred as it gives faster learning and better performance in the long run. However, in this specific case, it might be one of the reasons for the observed gap.

With $\lambda = 1$, on the metrics depicted in Figure 5.4, we observe that the performance of B improves respect $\lambda = 0.95$. Agent A still shows a faster learning speed, as it needs fewer timesteps to reach convergence. Despite still existing a clear gap between the two agents, it is smaller as a result of a slight decline in A’s survival time and a notable improvement in B’s. The “eat count per timestep” graph, depicted in Figure 5.3 (right), further shows this effect. In this case, B starts eating as soon as the episode begins, allowing the agent to extend its survival later. This is very likely driven by $\lambda = 1$, which ensures that the impact of longer episodes (higher return for B) does not decay in the early steps of the episodes.

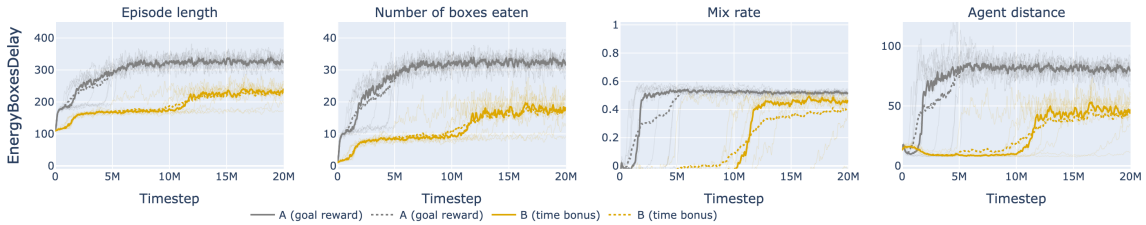


Figure 5.4: **EnergyBoxesDelay** with full Monte Carlo returns ($\lambda = 1$)

In summary, this environment is an example of a setting where the time-cost agent struggles compared to the standard goal-oriented design. While both agents can solve the

task, the delayed effects of actions in this environment make the goal-oriented reward a *better-shaped* learning signal for the agent.

EnergyBoxesHard In this environment, Agent B performs better than A, in terms of learning speed and stability as well as end performance.

The specific energy configuration is the following: initial energy is set at 5, eating replenishes 6 units and boxes refill immediately once the opposite box is eaten. Episodes are truncated after 512 timesteps. This *hard* setting only leaves room for one very specific survival strategy: the agent needs to switch between boxes using the minimal number of timesteps that MiniGrid actions and box positioning allow, which is 6. This means that, given the tight margin, any deviation from this optimal path will result in episode termination.

Both agents learn the optimal strategy, as both seem to occasionally achieve episode truncation early in the training process. However, Agent A learns slower and its learning curve is very noisy. Even when it reaches convergence, it looks clearly less stable than B, as it makes more mistakes that lead to early episode termination. On the other hand, B learns faster and more efficiently and after around 10M training steps, every individual run seems to consistently reach 512 steps, barely making mistakes.

Unlike in the delayed environment, given that any mistake leads to almost immediate episode termination, the reward functions of A and B should provide a very similar signal. In the case of A, any action deviating from the optimal path will not allow reaching the next box which means not getting the +1 (and possibly more in later timesteps). For B, making a mistake means that episode will terminate within the next timesteps and thus will only get a positive time bonus for a few steps. For that reason, we expected both agents to be very close in this setting, but we find that B actually surpasses A. One possible reason behind this is that the smoother reward function of B has a lower variance, which guides the agent more consistently towards the optimal policy.

In conclusion for this section, we can highlight some observations. The experiments demonstrate that agents can learn and solve tasks without explicit reward in environments where survival dynamics naturally encode the goal at hand. The main benefit of this approach is the lack of need for task-specific reward design, despite traditional rewards still performing better, in general. The degree to which survival dynamics encode the goals we want to solve determines the performance of these agents: in settings where the information encoded by the environment dynamics closely resembles the conventional reward, time-rewarded agents can match, or even surpass, as we observe, conventional goal-oriented agents.

Chapter 6

Conclusions and Future Work

In this thesis, we investigated how time can play a role in shaping and defining rewards for reinforcement learning. Our main goal was to understand if the agent’s lifetime could serve as a general reward, providing a domain-agnostic and adaptable framework that does not require task-specific knowledge. To this end, we wanted to answer research questions including how and where this idea could be useful in traditional reinforcement learning problems.

In our initial *soft* approach, we looked at how time could enhance reward shaping. The experiments showed satisfactory results: adding a small cost to time, in the form of a fixed penalty for every timestep taken on top of the original reward, proved to have significant effects with respect to the standard baselines. Some of the outcomes are interesting and counterintuitive, such as the fact that a standard binary reward (+1 for reaching the goal, 0 otherwise) can reach the same behaviour and performance as more involved reward designs which include a penalty for time. While the results were not groundbreaking, they hinted at the potential of using time in a more drastic way.

In our second *hard* approach, we explored the possibility of using lifetime as the sole reward, by providing a fixed reward for every timestep taken, assuming the termination dynamics of the environment inherently encode the goals for the agent. The experiments here show an interesting result: agents can learn in this setting without an explicit reward. We found that, in this setting, agents learn how to solve tasks and are close both in learning speed and end performance to the standard goal-based baseline. Moreover, unexpectedly, the time-based reward even performed better than the goal-based for some specific configurations.

The key takeaway from our work is the value of an agent’s lifetime when it comes to reward design, both as a basic reward shaper and as the main reward on its own. Given that the agent’s lifetime is general to any environment, the proposed methodologies are applicable to a broad range of problems without the need for task-specific reward design.

Moreover, the simplicity of this method suggests that it could be scaled to much bigger and more complex settings, although this still needs to be tested further, especially for environments where the relationship between termination dynamics and goals is not as strong.

On the other hand, our research also has a number of limitations. While the obtained results are interesting and relevant on their own, applying the hard approach to environments that do not directly fall into the “energy budget” category might be a challenge. Moreover, because of time and resource constraints, the experiments were only performed using a single learning algorithm (PPO) and grid world environments. This allows for a lot more room for exploration of these methodologies in more complex environments and with other learning algorithms.

Looking ahead, we identify several directions to further this research. Firstly, by testing the viability of our approach in more complex environments, such as Atari, ProcGen, and Mario Bros, to truly test its robustness in environments where the relationship between the termination probabilities and underlying goals is not as direct as in the tested environments. Secondly, while our experiments utilized the PPO learning algorithm, it would be interesting to explore how our methodologies work with other algorithms, extending even beyond the policy gradients family, in order to also identify which of the effects observed in our experiments can be attributed to the learning algorithm and which ones to the proposed methodologies themselves. Lastly, a significant extension would include exploring our approach in environments that do not inherently fit the *energy budget* paradigm.

List of Figures

2.1	The agent-environment interaction in reinforcement learning. Image source: ((Sutton and Barto (2018)))	8
2.2	Comparison of value-based and policy-based deep reinforcement learning. For value-based methods, given a state s , the network predicts a value estimate $Q(s, a)$ for each of the actions a_1, a_2 . In the case of policy-based, the network directly outputs the policy in the form of a probability distribution over the actions, given the state s	13
4.1	Render of 8 different MiniGrid environments	25
4.2	Agent’s architecture diagram	26
4.3	Environments used in Experiment I	27
4.4	Experiment I results on the following environments: MiniGrid-Empty-16x16, MiniGrid-SimpleCrossingS9N2, MiniGrid-FourRooms	29
4.5	Experiment I results on the following environments: MiniGrid-Unlock, MiniGrid-DoorKey-8x8, MiniGrid-KeyCorridorS3R2	31
4.6	Experiment I results on the EnergyBoxes-Empty-16x16 environment for different value of the GAE λ parameter. Note that the x-axis, representing the number of steps in the training process, is different for each plot, given the varying training speeds.	32
5.1	Grid of EnergyBoxes and EnergyBoxesHard environments used in Experiment II	36
5.2	Experiment II results on the following environments: EnergyBoxes, EnergyBoxesDelay, EnergyBoxesHard. Each of the background lines represents one individual run, then summarized by the median and mean.	38
5.3	EnergyBoxesDelay sum of boxes eaten per episode timestep; fully trained agents (aggregated episodes 1M timesteps)	40
5.4	EnergyBoxesDelay with full Monte Carlo returns ($\lambda = 1$)	40

Bibliography

- Abel, D., Allen, C., Arumugam, D., Hershkowitz, D. E., Littman, M. L., and Wong, L. L. (2021). Bad-policy density: A measure of reinforcement learning hardness. *arXiv preprint arXiv:2110.03424*.
- Barto, A., Lewis, R., and Singh, S. (2009). Where do rewards come from?
- Bellemare, M. G., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., and Munos, R. (2016). Unifying count-based exploration and intrinsic motivation.
- Chevalier-Boisvert, M., Dai, B., Towers, M., de Lazcano, R., Willems, L., Lahlou, S., Pal, S., Castro, P. S., and Terry, J. (2023). Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *CoRR*, abs/2306.13831.
- Colombetti, M., Dorigo, M., Borghi, G., and Jamshidi, M. (1996). Robot shaping: The hamster experiment.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor.
- Hadfield-Menell, D., Milli, S., Abbeel, P., Russell, S., and Dragan, A. (2020). Inverse reward design.
- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning.
- Kakade, S. and Langford, J. (2002). Approximately optimal approximate reinforcement learning.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.

- Moazami, S. and Doerschuk, P. (2020). Modeling survival in model-based reinforcement learning.
- Ng, A. Y., Harada, D., and Russell, S. J. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, ICML ’99, page 278–287, San Francisco, CA, USA. Morgan Kaufmann Publishers Inc.
- OpenAI (2022). Spinning up in deep rl: Proximal policy optimization. Accessed: 2023-06-07.
- OpenAI, Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., d. O. Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning.
- Raileanu, R. and Rocktäschel, T. (2020). Ride: Rewarding impact-driven exploration for procedurally-generated environments. In *International Conference on Learning Representations*.
- Randløv, J. and Alstrøm, P. (1998). Learning to drive a bicycle using reinforcement learning and shaping. In *International Conference on Machine Learning*.
- Ringstrom, T. J. (2022). Reward is not necessary: How to create a compositional self-preserving agent for life-long learning. arXiv:2211.10851.
- Rummery, G. and Niranjan, M. (1994). On-line q-learning using connectionist systems. *Technical Report CUED/F-INFENG/TR 166*.
- Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and Abbeel, P. (2017a). Trust region policy optimization.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2018). High-dimensional continuous control using generalized advantage estimation.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017b). Proximal policy optimization algorithms. arXiv:1707.06347.
- Singh, S., Lewis, R. L., Barto, A. G., and Sorg, J. (2010). Intrinsically motivated reinforcement learning: An evolutionary perspective. *IEEE Transactions on Autonomous Mental Development*, 2:70–82.

- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement learning: An introduction*. The MIT Press, 2nd edition.
- Sutton, R. S., Mcallester, D., Singh, S., and Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems 12*, volume 12, pages 1057–1063. MIT Press.
- van Hasselt, H., Guez, A., and Silver, D. (2015). Deep reinforcement learning with double q-learning.
- Wang, Z., Bapst, V., Heess, N., Mnih, V., Munos, R., Kavukcuoglu, K., and de Freitas, N. (2017). Sample efficient actor-critic with experience replay.
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3):279–292.
- Yoshida, N. (2016). On reward function for survival.
- Yu, T., Quillen, D., He, Z., Julian, R., Narayan, A., Shively, H., Bellathur, A., Hausman, K., Finn, C., and Levine, S. (2021). Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning.